

Analisis & Reverse Engineering Sistem Informasi Kasir (POS)

Mata Kuliah: Rekayasa Perangkat Lunak (RPL)

Jenis Tugas: Ujian Akhir Semester (UAS)

Anggota Kelompok:

1. Bayu
2. Muhammad Akbar
3. Fitria Rofiqoh

Studi Kasus: Audit logika bisnis dan rekonstruksi arsitektur backend pada sistem kasir berbasis Node.js + Express.

Tujuan: Memetakan alur backend, struktur data, dan menemukan celah keamanan serta rekomendasi perbaikan teknis.

Ringkasan Proyek & Metodologi

Metode Pengecekan:

- Membongkar dan menganalisis kode sumber aplikasi (*Reverse Engineering*).
- Cek alur Node.js / Express (rute API, pengolah data, dan model).
- Tes kode secara langsung dan simulasi transaksi di lingkungan uji coba.

Tujuan & Fokus Utama:

- Audit alur kerja sistem, menyusun ulang struktur *database*, dan mencari celah keamanan.
- Memastikan interaksi pengguna, aturan relasi antar data (minimal 5 tabel), dan keutuhan struk transaksi berjalan dengan benar.

Bukti Pengecekan: Titik Awal Aplikasi (File: src/index.ts)

```
import express from 'express';
import db from './db/sqlite.js';

const app = express();

app.use(express.json());

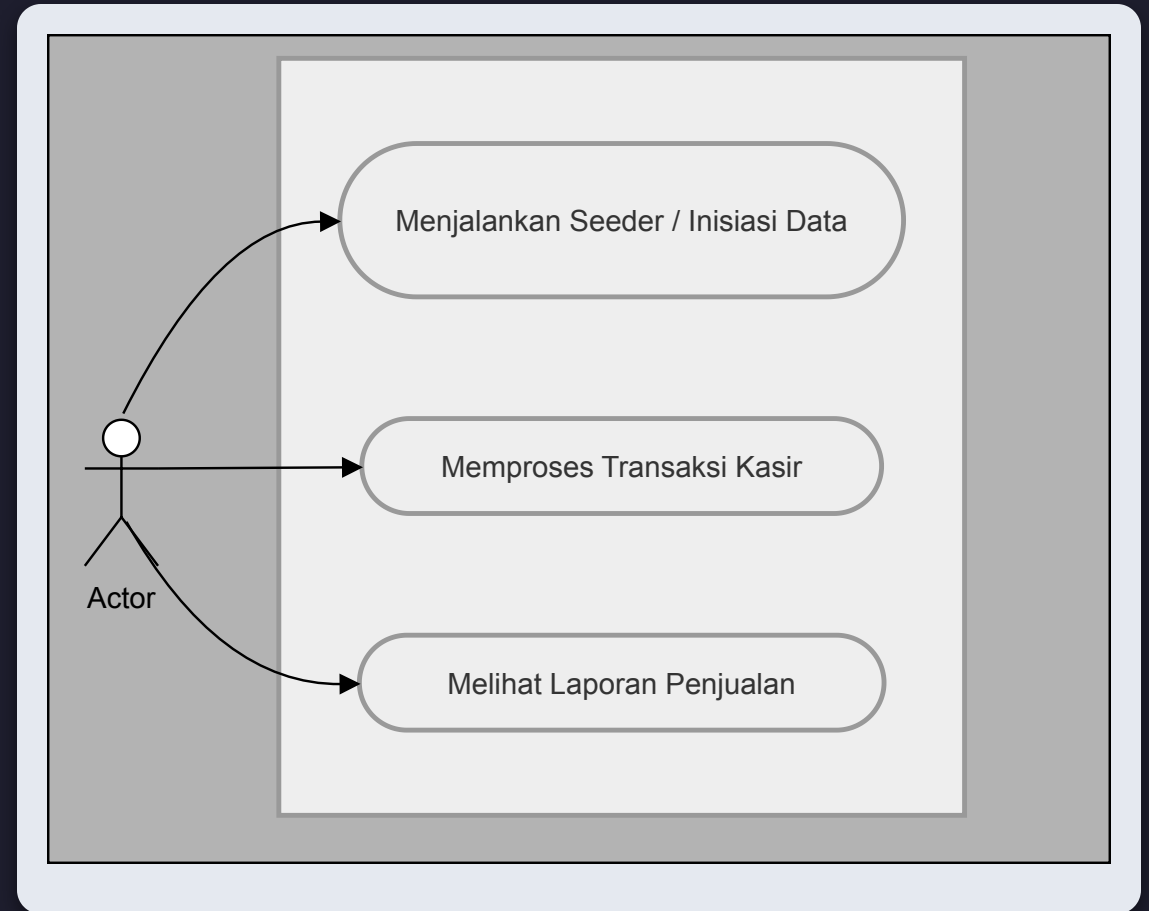
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server kasir berjalan lancar di http://localhost:${PORT}`);
});
```

Pemetaan Interaksi: Actor & Use Cases

Aktor Utama: Kasir (Staff Operasional)

Silakan klik pada salah satu proses di diagram untuk melihat detail bukti audit E2E, atau klik tombol di bawah untuk lanjut.

▶▶ Lewati Detail, Lanjut ke Struktur Database



Detail Audit: 1. Menjalankan Seeder



Kembali ke Diagram Use Case

Sisi Antarmuka (Aksi di Browser)

```
<button onclick="runSeed()">1. Jalankan Seeder (Isi Data Dummy)</button>
<script>
  function runSeed() { fetchAPI('/api/seed', 'POST'); }
</script>
```

Sisi Server (Eksekusi API di Node.js)

```
app.post('/api/seed', asyncHandler(async (req: Request, res: Response) => {
  await dbRun(`INSERT INTO users (name, role) VALUES (?, ?)`, ['Kasir Satu', 'Cashier']);
  await dbRun(`INSERT INTO categories (name) VALUES (?)`, ['Elektronik']);
  await dbRun(`INSERT INTO products ... VALUES (?, ?, ?)`, [1, 'Kabel Data', 25000]);

  res.status(201).json({ message: 'Data dummy berhasil di-generate!' });
}));
```

Detail Audit: 2. Memproses Transaksi Kasir



Kembali ke Diagram Use Case

Sisi Antarmuka (Aksi di Browser)

```
function runTx() {  
  fetchAPI('/api/transactions', 'POST', {  
    user_id: 1,  
    items: [ { product_id: 1, qty: 2 }, { product_id: 2, qty: 1 } ]  
  });  
}
```

Sisi Server (Penerimaan & Validasi di Node.js)

```
app.post('/api/transactions', asyncHandler(async (req: Request, res: Response) => {  
  const { user_id, items } = req.body as { user_id: number; items: any[] };  
  
  if (!user_id || !items || items.length === 0) {  
    throw new AppError('Data transaksi tidak lengkap.', 400);  
  }  
})
```

Detail Audit: 3. Melihat Laporan Penjualan



Kembali ke Diagram Use Case

Sisi Antarmuka (Aksi di Browser)

```
function runReport() {  
  fetchAPI('/api/reports/sales', 'GET');  
}
```

Sisi Server (Perakitan Laporan di Node.js)

```
app.get('/api/reports/sales', asyncHandler(async (req: Request, res: Response) => {  
  const query = `  
    SELECT t.transaction_date, u.name AS cashier_name, p.name AS product_name...  
    FROM transactions t JOIN users u ON t.user_id = u.id ...  
  `;  
  const reports = await dbAll(query);  
  res.status(200).json({ data: reports });  
}));
```


Struktur Database: Rekonstruksi Skema 5 Tabel

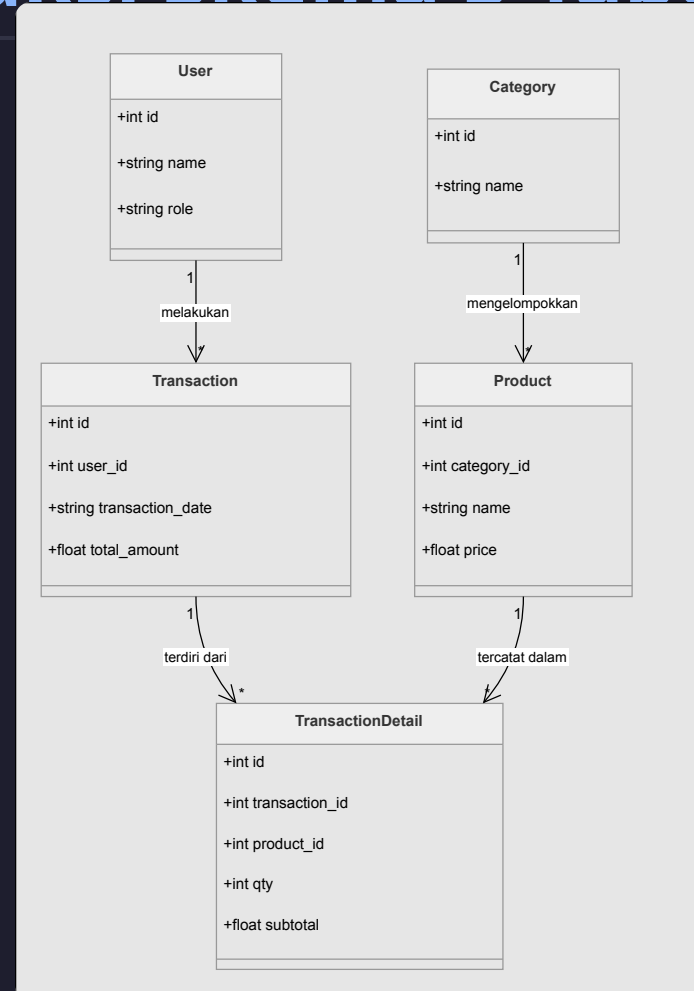
Prinsip Desain:

Normalisasi mendekati 3NF untuk menjaga konsistensi data historis.

Detail Audit:

Klik pada salah satu entitas di tabel untuk melihat bukti audit implementasi kodenya.

 Lewati Detail, Lanjut



Detail Skema: Entitas User

 **Kembali ke Class Diagram**

```
// Layer Aplikasi (src/types/index.ts)
export interface User { id?: number; name: string; role: 'Admin' | 'Cashier'; }

// Layer Database (src/db/sqlite.ts)
db.run(`CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  role TEXT CHECK(role IN ('Admin', 'Cashier')) NOT NULL
)`);
```

Detail Skema: Entitas Category

 **Kembali ke Class Diagram**

```
// Layer Aplikasi
export interface Category { id?: number; name: string; }

// Layer Database
db.run(`CREATE TABLE IF NOT EXISTS categories (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL
)`);
```

Detail Skema: Entitas Transaction

 **Kembali ke Class Diagram**

```
// Layer Aplikasi
export interface Transaction { id?: number; user_id: number; transaction_date: string; total_amount: number; }

// Layer Database
db.run(`CREATE TABLE IF NOT EXISTS transactions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  transaction_date TEXT NOT NULL,
  total_amount REAL NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users (id)
)`);
```

Detail Skema: Entitas Product

 [Kembali ke Class Diagram](#)

```
// Layer Aplikasi
export interface Product { id?: number; category_id: number; name: string; price: number; }

// Layer Database
db.run(`CREATE TABLE IF NOT EXISTS products (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  category_id INTEGER NOT NULL,
  name TEXT NOT NULL,
  price REAL NOT NULL,
  FOREIGN KEY (category_id) REFERENCES categories (id)
)`);
```

Detail Skema: Entitas TransactionDetail

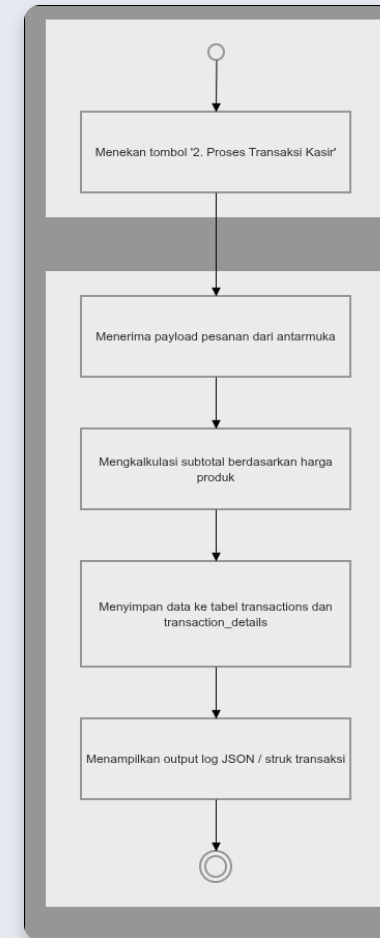
 **Kembali ke Class Diagram**

```
// Layer Aplikasi
export interface TransactionDetail { id?: number; transaction_id: number; product_id: number; qty: number; subtotal: number; }

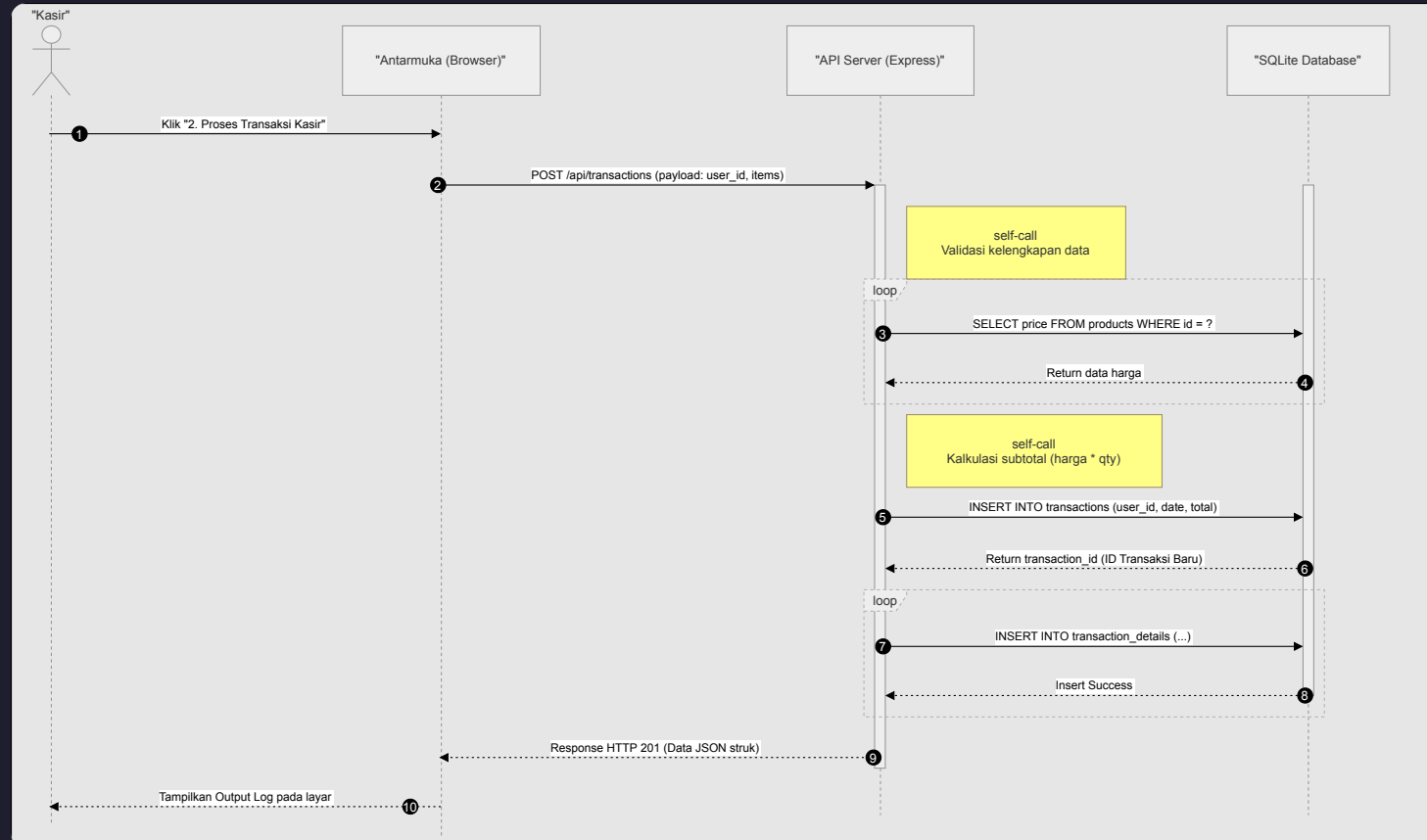
// Layer Database
db.run(`CREATE TABLE IF NOT EXISTS transaction_details (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  transaction_id INTEGER NOT NULL,
  product_id INTEGER NOT NULL,
  qty INTEGER NOT NULL,
  subtotal REAL NOT NULL,
  FOREIGN KEY (transaction_id) REFERENCES transactions (id),
  FOREIGN KEY (product_id) REFERENCES products (id)
)`);
```

Logika Alur Kerja Transaksi: Activity Diagram

- **1. Inisiasi:** Aktor menekan tombol proses transaksi.
- **2. Penerimaan:** Sistem menerima *payload* pesanan dari antarmuka.
- **3. Kalkulasi:** Mengkalkulasi subtotal berdasarkan harga produk (Validasi Anti-Fraud).
- **4. Penyimpanan:** Menyimpan data secara atomik ke tabel `transactions` dan `transaction_details`.
- **5. Output:** Menampilkan *log* **JSON** atau struk transaksi ke layar kasir.



Alur Kerja Transaksi: Sequence Diagram



▶▶ Lanjut ke Temuan Kritis Audit

Detail Sequence: 1. Penerimaan Payload



Kembali ke Sequence Diagram

Fokus: Aktor mengirim data belanjaan, lalu **Express** memvalidasi keberadaan `user_id` dan `items`.

```
// Lifeline Antarmuka (Klien) mengirim pesan
fetchAPI('/api/transactions', 'POST', { user_id: 1, items: [...] });

// Lifeline API Server (Backend) menerima pesan
app.post('/api/transactions', asyncHandler(async (req: Request, res: Response) => {
  const { user_id, items } = req.body;
})));
```

Detail Sequence: 2. Validasi Harga



Kembali ke Sequence Diagram

Fokus: Looping untuk mengecek harga asli di `database` guna mencegah manipulasi payload dari sisi klien.

```
for (const item of items) {  
  // Panah Solid: API meminta data ke Database  
  const products = await dbAll(  
    `SELECT price FROM products WHERE id = ?`, [item.product_id]  
  );  
  
  // Panah Putus-putus: Database mengembalikan data ke API  
  const price = products[0].price;  
}
```

Detail Sequence: 3. Insert Header Transaksi

 **Kembali ke Sequence Diagram**

Fokus: Pembuatan record nota utama di tabel `transactions`.

```
// Panah Solid: API mengirim perintah Insert
const insertTx = await dbRun(
  `INSERT INTO transactions (user_id, transaction_date, total_amount) VALUES (?, ?, ?)`,
  [user_id, dateNow, total_amount]
);

// Panah Putus-putus: API menangkap ID yang dikembalikan Database
const transaction_id = insertTx.id;
```

Detail Sequence: 4. Insert Detail Item

 **Kembali ke Sequence Diagram**

Fokus: Looping untuk menyimpan setiap barang yang dibeli ke tabel `transaction_details`

```
for (const detail of processedItems) {  
  // Panah Solid: API mengirim perintah Insert Detail  
  await dbRun(  
    `INSERT INTO transaction_details (transaction_id, product_id, qty, subtotal) VALUES (?, ?, ?, ?)`,  
    [transaction_id, detail.product_id, detail.qty, detail.subtotal]  
  );  
  // Panah Putus-putus (Success) terjadi secara implisit jika tidak ada error  
}
```

Detail Sequence: 5. Response ke Klien

 **Kembali ke Sequence Diagram**

Fokus: Pengembalian status kode **201 Created** beserta metadata struktur JSON sebagai bukti transaksi berhasil.

```
// Panah Putus-putus panjang: API Server membalas ke Antarmuka  
res.status(201).json({  
  message: 'Transaksi berhasil diproses.',  
  data: { transaction_id, total_amount, date: dateNow }  
});
```

Temuan Kritis: Hasil Audit & Reverse Engineering

Berdasarkan penelusuran *source code* dan pengujian E2E, kami menemukan **3 celah kritis** yang berisiko jika sistem dinaikkan ke tahap *Production*:

1. **Integritas Data (ACID) Terancam**

Tidak ada jaminan transaksi *database* aman dari kegagalan sistem (*Partial-Write*).

 [Lihat Bukti Kode](#)

2. **Keamanan Autentikasi Rentan (Forgery)**

Identitas kasir dikirim secara polos di *body request*, sangat mudah dimanipulasi.

 [Lihat Bukti Kode](#)

3. **Inventori & Atomicity Hilang**

Sistem mengabaikan arus barang dan hanya mencatat arus uang.

 [Lihat Bukti Kode](#)

 [Lewati Detail, Lanjut ke Rekomendasi](#)

Detail Temuan: 1. Integritas Data (ACID)



Kembali ke Daftar Temuan

Opini Auditor: Operasi *database* di bawah ini berjalan independen. Jika server tiba-tiba *crash* di tengah perulangan (`for loop`), nota utama sudah tercatat namun detail barang hilang sebagian (**Partial Write**).

Bukti Source Code (src/index.ts):

```
// Insert ke tabel transactions
const dateNow = new Date().toISOString();
const insertTx = await dbRun(
  `INSERT INTO transactions (user_id, transaction_date, total_amount) VALUES (?, ?, ?)`,
  [user_id, dateNow, total_amount]
);
const transaction_id = insertTx.id;

// Insert ke tabel transaction_details
for (const detail of processedItems) {
  await dbRun(
    `INSERT INTO transaction_details (transaction_id, product_id, qty, subtotal) VALUES (?, ?, ?, ?)`,
    [transaction_id, detail.product_id, detail.qty, detail.subtotal]
  );
}
```

Detail Temuan: 2. Keamanan Autentikasi

 [Kembali ke Daftar Temuan](#)

Opini Auditor: Variabel `user_id` diterima mentah-mentah tanpa verifikasi **Token JWT** atau middleware auth. Siapapun yang paham endpoint ini bisa memanipulasi payload dan melakukan transaksi atas nama staf kasir lain.

Bukti Source Code (src/index.ts):

```
// 2. MAIN FLOW: TRANSAKSI KASIR
app.post('/api/transactions', asyncHandler(async (req: Request, res: Response) => {
  const { user_id, items } = req.body as { user_id: number; items: { product_id: number; qty: number }[] };

  if (!user_id || !items || items.length === 0) {
    throw new AppError('Data transaksi tidak lengkap. Pastikan user_id dan items terisi.', 400);
  }
});
```


Detail Temuan: 3. Inventori & Atomicity



Kembali ke Daftar Temuan

Opini Auditor: Sistem saat ini baru sebatas mesin penghitung uang. Secara struktural, entitas `products` mengabaikan ketersediaan barang di rak (tidak ada kolom `stock`), sehingga mustahil membangun fitur pengurangan stok otomatis (**Atomicity**).

Bukti Source Code (src/db/sqlite.ts):

```
// 3. Create products table
db.run(`CREATE TABLE IF NOT EXISTS products (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  category_id INTEGER NOT NULL,
  name TEXT NOT NULL,
  price REAL NOT NULL,
  FOREIGN KEY (category_id) REFERENCES categories (id)
)`);
```

Rekomendasi Teknis Prioritas

Berdasarkan temuan, berikut adalah **perbaikan mutlak** sebelum rilis ke *Production*:

1. **Implementasi DB Transactions** (`BEGIN`, `COMMIT`, `ROLLBACK`) untuk menjamin ACID.
2. **Ganti `user_id` polos dengan JWT** untuk verifikasi identitas kasir.
3. **Penambahan mekanisme stok atomik** dan *audit log immutable*.

Simulasi Rekomendasi Perbaikan (DB Transaction):

```
// ❌ BEFORE: Rentan Partial-Write jika server crash di tengah proses
await dbRun(`INSERT INTO transactions ...`);
await dbRun(`INSERT INTO transaction_details ...`);
```

```
// ✅ AFTER: Aman dengan dibungkus DB Transaction
await dbRun('BEGIN TRANSACTION');
try {
  const tx = await dbRun(`INSERT INTO transactions ...`);
  await dbRun(`INSERT INTO transaction_details ...`);
  await dbRun('COMMIT'); // Data tersimpan sempurna
} catch (error) {
  await dbRun('ROLLBACK'); // Batalkan semua ke state awal jika ada 1 yang gagal
  throw error;
}
```

Evaluasi Reporting & Kelayakan Produksi

Fungsi laporan penjualan (`/api/reports/sales`) secara logika sudah bekerja untuk ekstraksi data, namun:

1. **Implementasi DB Transactions** (`BEGIN` , `COMMIT` , `ROLLBACK`) untuk menjamin ACID.
2. **Ganti `user_id` polos dengan JWT** untuk verifikasi identitas kasir.
3. **Penambahan mekanisme stok atomik** dan *audit log immutable*.

Simulasi Rekomendasi Perbaikan (Database Indexing):

```
// ❌ BEFORE: Tabel polos, query laporan akan lambat jika data jutaan  
db.run(`CREATE TABLE transactions ( ... transaction_date TEXT ... )`);
```

```
// ✅ AFTER: Optimalisasi dengan Indexing untuk kecepatan Query Laporan  
db.run(`CREATE TABLE transactions ( ... transaction_date TEXT ... )`);  
  
// Mempercepat proses ORDER BY t.transaction_date DESC pada laporan  
db.run(`CREATE INDEX idx_transaction_date ON transactions(transaction_date)`);  
  
// Mempercepat proses JOIN pada transaction_details  
db.run(`CREATE INDEX idx_td_product ON transaction_details(product_id)`);  
}
```

Kesimpulan & Langkah Implementasi

Kesimpulan Utama Audit:

- **Skema 5 tabel** sudah kuat dan sesuai prinsip relasional (layak untuk produksi setelah perbaikan).
- **Perbaikan mutlak:** transaksi DB atomik, otentikasi JWT, dan pengelolaan stok otomatis.

Rekomendasi Roadmap (Langkah Selanjutnya):

1. Implement **transaksi & stok atomik**.
2. Terapkan **JWT** + *role-based checks*.
3. Audit log **immutable**.
4. Refactor kode untuk **testability** dan **monitoring**.

Penutup

Sebagai penutup, hasil audit dan *reverse engineering* kami menyimpulkan bahwa sistem kasir ini sudah **memiliki pondasi struktur data yang baik**.

Namun, untuk layak naik ke tahap produksi, perbaikan pada aspek keamanan dengan **JWT** dan konsistensi data dengan **DB Transactions** adalah hal yang wajib dikejar terlebih dahulu.

Sekian presentasi dari kelompok kami, mohon maaf bila ada kekurangan.
Terima kasih.

TERIMA KASIH !

Sesi Tanya Jawab (Q&A) Dibuka

Tim Auditor: Bayu, Muhammad Akbar dan Fitria Rofiqoh